UAI
Universidad Abierta Interamericana

FACULTAD DE TECNOLOGÍA INFORMÁTICA
Sede Centro



TRABAJO PRÁCTICO INDIVIDUAL · 2026

Generala

Documentación técnica del juego de dados para dos jugadores en C# y SQL Server

ALUMNA
Valentina Morana

MATERIA
Desarrollo y Arquitectura de Software

COMISIÓN
2B

DOCENTE
Christian Parkinson

SEDE
Centro

FECHA
Segundo Cuatrimestre

Índice

1. Consigna del trabajo práctico..... 3
1.1 Requisitos LUG (comunes a todos los TP individuales)..... 3
2. Elección del juego..... 3
3. Reglas de negocio implementadas..... 4
3.1 Modalidad de juego..... 4
3.2 Mecánica de un turno..... 4
3.3 Cálculo de puntaje por categoría..... 4
3.4 Fin de partida y abandono..... 5
4. Arquitectura en capas..... 6
5. Clases principales por capa..... 7
5.1 BE: Entidades..... 7
5.2 DAL: Acceso a datos..... 7
5.3 BLL: Lógica de negocio..... 8
5.4 GUI: Presentación..... 8
6. Modelo de datos..... 10
7. Funcionalidades implementadas..... 11
7.1 Juego..... 11
7.2 Usuarios y sesión..... 12
7.3 Registro y persistencia..... 12
7.4 Robustez..... 12
7.5 Pruebas unitarias..... 13
8. Bitácora, XML de movimientos y estadísticas..... 13
8.1 Bitácora..... 13
8.2 Registro de movimientos en XML..... 13
8.3 Estadísticas..... 14
9. Diagramas de clases..... 15
10. Imágenes del sistema..... 23
10.1 Inicio de sesión..... 23
10.3 Pantalla de juego..... 24
10.4 Partida en curso..... 25
10.5 Reglas del juego..... 26
10.6 Estadísticas..... 26
10.7 Movimientos de la partida..... 27
10.8 Abandonar la partida..... 27
10.9 Bitácora..... 28
10.10 Mensajes y confirmaciones..... 28
11. Instalación y ejecución..... 29
11.1 Requisitos..... 29
11.2 Pasos..... 29

1. Consigna del trabajo práctico

De la lista de juegos propuesta por la cátedra se eligió la **Generala**. El enunciado es:

Desarrollar el juego de la Generala para 2 jugadores. En el cual se deberá administrar los turnos de cada jugador, conocer el puntaje y el ganador. Calcular los valores resultantes de los tiros de los 5 dados según la combinatoria obtenida (Generala, Full, Póker, etc.).

1.1 Requisitos LUG (comunes a todos los TP individuales)

- Aplicativo orientado a objetos programado en C#.
- Base de datos MS SQL Server.
- Arquitectura de 4 capas como mínimo.
- Registro de usuarios con credenciales.
- Login / Logout de jugadores.
- Bitácora de inicios de sesión, cierres de sesión, comienzo y finalización de partida.
- Registro de estadísticas de partidas ganadas, perdidas y empatadas.
- Registro en un archivo XML de todos los movimientos de la partida, para su posterior consulta.
- Promedio de victorias del jugador.
- Total acumulado de tiempo jugado por el jugador.
- Backup y Restore de la base de datos.

2. Elección del juego

Se eligió la Generala porque sus reglas son acotadas: 11 categorías fijas, 3 tiradas por turno y ningún tablero que administrar. Además, el puntaje de cada categoría es un cálculo sobre los 5 dados que no depende de la base ni de la interfaz, por lo que queda aislado en la BLL.



3. Reglas de negocio implementadas

3.1 Modalidad de juego

- 2 jugadores humanos en la misma PC y la misma ventana (modalidad “pasar y jugar”). No hay oponente automático ni juego en red.
- Ambos jugadores deben estar registrados y autenticados. El Jugador 1 inicia sesión al abrir la aplicación; el Jugador 2 lo hace desde la pantalla de juego con *Ingresar Jugador 2*, y debe ser un usuario distinto del Jugador 1: si se intenta con el mismo usuario, se rechaza antes de validar las credenciales, así no queda un inicio de sesión falso en la bitácora.
- Después del login se abre el menú principal (`FormMenu`), con las opciones Jugar, Estadísticas, Bitácora, Reglas y Cerrar sesión. *Jugar* abre la pantalla de juego y el botón *Menú* regresa al menú sin cerrar la sesión.
- En una revancha (volver a elegir *Jugar* desde el menú) se reutiliza el mismo Jugador 2 sin volver a loguearse, porque el menú lo recuerda. Si ingresa un Jugador 2 distinto, al anterior se le registra el cierre de sesión en la bitácora.
- Para registrarse, el nombre de usuario debe tener entre 3 y 20 caracteres y no puede tener espacios; la contraseña, entre 4 y 20 caracteres y al menos un número.

3.2 Mecánica de un turno

1. El jugador tiene hasta 3 tiradas de los 5 dados.
2. Después de cada tirada elige, con una casilla por dado, cuáles conservar; los no marcados se vuelven a tirar. Las casillas de “guardar” solo se habilitan después de la primera tirada del turno.
3. En cualquier momento posterior a la primera tirada, elige una categoría todavía no anotada y anota el puntaje que corresponde a los dados actuales.
4. Si los dados no cumplen la condición de la categoría elegida, se anota 0 y la categoría queda tachada.
5. Antes de anotar un 0 la aplicación pide confirmación; si el jugador responde que no, no se anota nada y puede elegir otra categoría.
6. Tachar a voluntad: con el botón *Tachar*, el jugador puede anotar 0 en cualquier categoría disponible aunque los dados den puntos, para descartar una que no le conviene jugar. También se pide confirmación, y la categoría ya no puede volver a usarse.
7. Desde la segunda tirada del turno, si no se guardó ningún dado, la aplicación pide confirmación antes de volver a tirar los 5.
8. Al anotar, el turno pasa automáticamente al otro jugador.

3.3 Cálculo de puntaje por categoría

Cada jugador completa sus propias 11 categorías. La tirada servida (combinación lograda en la primera tirada del turno) duplica el puntaje de las cuatro combinaciones, y la aplicación lo informa con un mensaje.

Tabla 1. Categorías y puntajes

Categoría	Condición	Puntos	Servida
Unos a Seises	Suma de los dados que muestran ese número	Variable	No aplica
Escalera	1-2-3-4-5 o 2-3-4-5-6	20	40
Full	Exactamente 3 dados de un valor y 2 de otro	30	60
Póker	4 o más dados iguales	40	80
Generala	Los 5 dados iguales	50	100
Generala Doble	5 dados iguales, habiendo anotado antes la Generala **con puntos**	100	No aplica

Si la Generala quedó tachada (0), la Generala Doble sigue disponible para que ambos jugadores completen siempre 11 turnos, pero solo puede tacharse: vale 0 aunque salgan 5 dados iguales.

3.4 Fin de partida y abandono

- La partida termina cuando ambos jugadores anotaron las 11 categorías. Gana quien suma más puntos; si los totales son iguales, es empate.
- La partida puede abandonarse en curso: la aplicación pregunta qué jugador abandona (cualquiera de los dos, le toque o no el turno), se guardan los puntajes parciales y la partida la gana siempre el otro jugador.
- Si se vuelve al menú (botón *Menú* o la X de la pantalla de juego) con una partida en curso, se pide elegir qué jugador abandona y la partida se registra como abandonada por ese jugador; si se cancela, la partida sigue. Sin partida en curso, se vuelve directamente al menú. Para salir de la aplicación se usa *Cerrar sesión* o la X del menú, y en los dos casos se pide confirmación.
- En ambos casos la BLL cierra la partida, guarda el resultado y el fin en la bitácora en una sola transacción (si esa transacción falla, la partida sigue en curso y se puede volver a intentar), registra el cierre en el XML de movimientos y notifica a la interfaz mediante el evento `PartidaFinalizada`. El texto del resultado (ganador, empate o quién abandonó) lo arma la BLL con `MensajeFinal`.

4. Arquitectura en capas

La solución está organizada en 4 proyectos de Visual Studio físicamente separados: tres bibliotecas de clases (BE, DAL, BLL) y el ejecutable de Windows Forms (GUI). Las referencias siguen la cadena **GUI → BLL → DAL**, y las tres capas usan las entidades de **BE**, que no depende de ninguna otra. La GUI nunca referencia a la DAL ni ejecuta SQL. Además, la solución incluye un quinto proyecto, `Tests.Unitarios`, con pruebas automáticas de la BLL (ver sección 7.5); no forma parte del ejecutable ni es una capa del sistema.

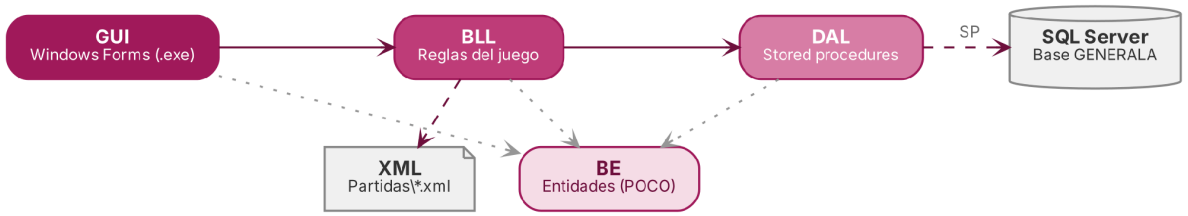


Figura 1. *Capas del sistema y sus dependencias*

Capa	Responsabilidad
BE (Business Entities)	Clases planas (POCO) que viajan entre capas: usuarios, partidas, registros de bitácora, movimientos del XML, estadísticas y las filas que se muestran en las grillas. No tienen lógica, solo propiedades.
DAL (Data Access Layer)	Encapsula todo el acceso a SQL Server y al archivo XML de movimientos. `ACCESO` (**internal**: solo la usan los mappers) maneja la conexión, las **transacciones** y la ejecución de stored procedures. Cada tabla tiene su mapper (`MP_USUARIO`, `MP_PARTIDA`, `MP_LOG`), que hereda de la clase abstracta y genérica **MAPPER<T>**; el XML lo lee y escribe `MOVIMIENTOXML`. Nunca se concatena SQL con datos de la interfaz, y hacia arriba solo se exponen objetos BE.
BLL (Business Logic Layer)	Todas las reglas del juego y el **estado de la partida**: turnos, tiradas, dados guardados, tirada servida, planilla de puntajes, ganador, abandono y XML de movimientos (PARTIDA); puntaje de las 11 categorías (GENERALA); estadísticas y filtro del historial (ESTADISTICA); registro con sus reglas, login, logout y bitácora. También arma los textos del juego que muestra la interfaz (resultado final, aviso antes de tachar y quién va ganando). Si la base de datos devuelve un error, lo convierte en una excepción con un mensaje claro. Notifica el fin de partida con un evento.
GUI (Presentación)	Windows Forms. Los formularios **no guardan estado del juego ni hacen cálculos**: muestran lo que les devuelve la BLL, le avisan lo que hace el usuario y muestran los errores. Después del login se navega desde un menú (`FormMenu`). Los dados son un control de usuario propio (`ucDado`), los mensajes usan un cuadro de diálogo propio (`FormMensaje`) y los colores están centralizados en la clase `Tema`.

5. Clases principales por capa

5.1 BE: Entidades

Clase	Descripción
USUARIO	ID, nombre y contraseña de un jugador.
PARTIDA	Los 2 jugadores, el ganador, los puntajes finales, las fechas de inicio y fin y la ruta del XML de la partida.
FILAPUNTAJE	Fila de la planilla de puntajes (categoría y puntaje de cada jugador).
FILAHISTORIAL	Fila del historial de estadísticas (fecha, rival, puntajes, resultado y ruta del XML).
MOVIMIENTO	Un movimiento leído del XML de la partida (tirada, anotación o fin): turno, jugador, acción, dados, dados guardados, categoría o resultado, puntaje y fecha.
LOG	Un registro de la bitácora: usuario, tipo de evento y descripción; al listar la bitácora también trae la fecha y los nombres del usuario y del tipo de evento. Tiene las constantes de los 4 tipos de evento, que usan todas las capas.
ESTADISTICA	Resultado de las estadísticas de un jugador: ganadas, perdidas, empatadas, promedio de victorias, tiempo total y una lista de `FILAHISTORIAL` (agregación).

5.2 DAL: Acceso a datos

Clase	Descripción
ACCESO	Clase **internal**. Abre y cierra la conexión, maneja la transacción (`IniciarTx`, `AceptarTx`, `CancelarTx`) y ejecuta los stored procedures: `Escribir`, `LeerEscalar` y `Leer`, que devuelve un `DataTable`. La cadena de conexión se lee de `App.config`; si ese servidor no responde, `ResolverServidor` prueba `.\SQLEXPRESS` y la instancia por defecto, y recuerda el que funcionó.
MAPPER<T>	Clase **abstracta y genérica**, base de todos los mappers. Tiene la referencia a `ACCESO` y declara el método abstracto `Insertar(T)`, que cada mapper implementa con override.
MP_USUARIO	`MAPPER<BE.USUARIO>`: alta (devuelve 0 si el nombre ya existe), login y búsqueda por ID.
MP_PARTIDA	`MAPPER<BE.PARTIDA>`: alta y cierre de la partida, cada uno en **una transacción** junto con los registros de bitácora de los dos jugadores; listado de partidas terminadas de un usuario.
MP_LOG	`MAPPER<BE.LOG>`: inserción de un registro en la bitácora y listado completo (`Listar`, con `BITACORA_LISTAR`), del más nuevo al más viejo.
MOVIMIENTOXML	Lee y escribe el archivo XML de cada partida: lo crea con los dos jugadores, inserta los nodos Tirada, Movimiento y Fin, y devuelve la lista de `BE.MOVIMIENTO` para consultarla.
BACKUPRESTORE	BACKUP DATABASE / RESTORE DATABASE WITH REPLACE usando la cadena MASTER del `App.config`; obtiene la carpeta de backups de la instancia y devuelve el mensaje real de SQL Server si algo falla. `HacerRestore` también informa si los datos llegaron a restaurarse (parámetro `out restaurada`), para que la aplicación vuelva al login.

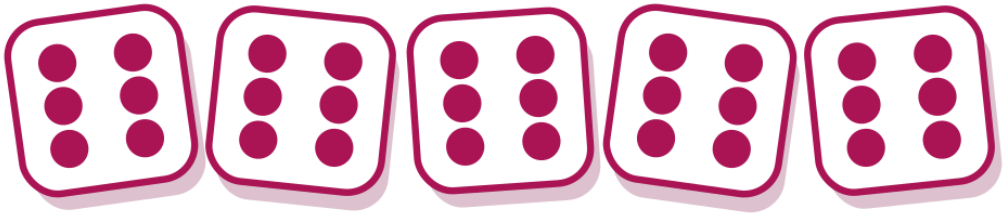
5.3 BLL: Lógica de negocio

Clase	Descripción
GENERALA	Tira los dados respetando los guardados y calcula el puntaje de las 11 categorías, incluida la Generala Doble y el bonus por tirada servida.
PARTIDA	Maneja la partida completa y **su estado**: jugadores, turno, tiradas restantes, dados y planilla. `Iniciar`, `Tirar` (registra la tirada en el XML), `CategoriasDisponibles`, `CalcularPuntaje`, `Anotar` y `Tachar` (registran la anotación en el XML y pasan el turno), `Abandonar(quienAbandona)` y el cierre con el ganador (registra el fin en el XML). Informa el turno del jugador (`TurnoDelJugador`, de 1 a 11) y quién va ganando (`JugadorQueVaGanando`, `AmbosAnotaron`, `MensajeVaGanando`), y arma los textos que muestra la interfaz (`AvisoAntesDeAnotar`, `MensajeFinal`). Dispara el evento `PartidaFinalizada`.
delPartidaFinalizada	Delegado del evento `PartidaFinalizada`; `FormPrincipal` se suscribe para mostrar el resultado.
ESTADISTICA	Calcula las estadísticas de un jugador a partir de sus partidas terminadas (usa `MP_PARTIDA` y `MP_USUARIO`): ganadas, perdidas, empatadas, promedio de victorias, tiempo total e historial. `Filtrar` devuelve solo las partidas del historial con el resultado elegido (enum `FILTRO_RESULTADO`: Todas, Ganadas, Perdidas, Empates).
USUARIO	Registro, login y logout, con el registro de inicio y cierre de sesión en la bitácora. **ValidarRegistro** aplica las reglas de alta (nombre de 3 a 20 caracteres sin espacios; contraseña de 4 a 20 caracteres con al menos un número). Distingue un nombre repetido de un error de base de datos.
BITACORA	Registra el inicio y el cierre de sesión mediante `MP_LOG`, y `Listar` devuelve todos los registros para mostrarlos. El inicio y el fin de partida se graban dentro de la transacción de la partida.
MOVIMIENTOXML	Arma los datos de cada registro del XML de la partida: la tirada (`RegistrarTirada`), la anotación (`RegistrarMovimiento`) y el cierre (`RegistrarFin`), con la fecha y los dados como texto. La lectura y escritura del archivo las hace `DAL.MOVIMIENTOXML`; `LeerMovimientos` devuelve los movimientos para consultarlos, y `Resolver` encuentra el archivo aunque la partida se haya jugado en otra PC.
BACKUPRESTORE	Envoltorio sobre la clase homónima de la DAL, incluida la carpeta de backups por defecto.

5.4 GUI: Presentación

Formulario	Descripción
FormLogin / FormRegistro	Ingreso y alta de usuarios. `FormLogin` se reutiliza para el ingreso del Jugador 2 y rechaza al usuario del Jugador 1; Enter ingresa y el enlace Registrate abre `FormRegistro`, que valida las reglas de alta antes de guardar.
FormMenu	Menú que aparece después del login: Jugar, Estadísticas, Bitácora, Reglas y Cerrar sesión. Recuerda al Jugador 2 entre partidas (revancha). Su evento `FormClosing` registra el logout de los dos jugadores por separado (si falla el de uno, igual se registra el del otro), tanto con Cerrar sesión como al cerrar la aplicación (con confirmación).

Formulario	Descripción
FormPrincipal	Pantalla de juego: dados, selector de categoría (muestra cuánto daría cada una con los dados actuales), botones Anotar y Tachar, grilla de puntajes, turno del jugador ("Turno X de 11"), abandono, backup/restore y el botón Menú, que vuelve al menú. **No guarda estado**: después de cada acción actualiza la pantalla según `BLL.PARTIDA`. Pide confirmación antes de tachar una categoría y antes de volver a tirar sin dados guardados. Atajos: T tira los dados y Enter anota. Su evento `FormClosing` exige abandonar la partida en curso antes de volver al menú.
ucDado	Control de usuario: un dado (el valor que salió) con su casilla Guardar. `FormPrincipal` usa cinco (composición).
FormAbandonar	Pregunta qué jugador abandona la partida (botón Abandonar o volver al menú con una partida en curso) y devuelve el elegido a `FormPrincipal`. Con Cancelar, la partida sigue.
FormEstadisticas	Muestra el historial y los indicadores que calcula `BLL.ESTADISTICA` (ganadas/perdidas/empatadas, promedio de victorias y tiempo total), un filtro del historial por resultado y el botón Ver movimientos.
FormMovimientos	Muestra en una grilla los movimientos guardados en el XML de la partida elegida: tiradas, anotaciones (en negrita) y el cierre (resaltado). Arriba muestra el nombre del archivo, y la ruta completa aparece al pasar el mouse. Con el botón Descargar XML guarda una copia del archivo original en la carpeta que elija el usuario.
FormBitacora	Lista la bitácora completa (fecha y hora, usuario, evento y descripción), del más nuevo al más viejo, con los datos que devuelve `BLL.BITACORA.Listar`.
FormReglas	Ayuda con las reglas del juego.
FormMensaje / Mensaje	Cuadro de diálogo propio con los colores de la aplicación, que reemplaza a `MessageBox` para que los mensajes se vean igual en cualquier PC. `Mensaje.Show` tiene las mismas firmas que `MessageBox.Show`.
Tema	Clase estática con los colores de la aplicación (`Rosa`, `RosaClaro`, `Vino`), para cambiarlos en un solo lugar.



¡GENERALA!

6. Modelo de datos

La base de datos **GENERALA** se crea con el script `Script.sql`, que puede ejecutarse más de una vez sin errores: crea solo las tablas, datos iniciales y procedimientos que falten. Los textos (nombre, contraseña, descripción de la bitácora y ruta del XML) son nvarchar, para no perder caracteres no latinos; si la base ya existía con varchar, el script la migra.

DIAGRAMA 8

DER · Base GENERALA

Tablas, claves y relaciones de SQL Server, según Script.sql.

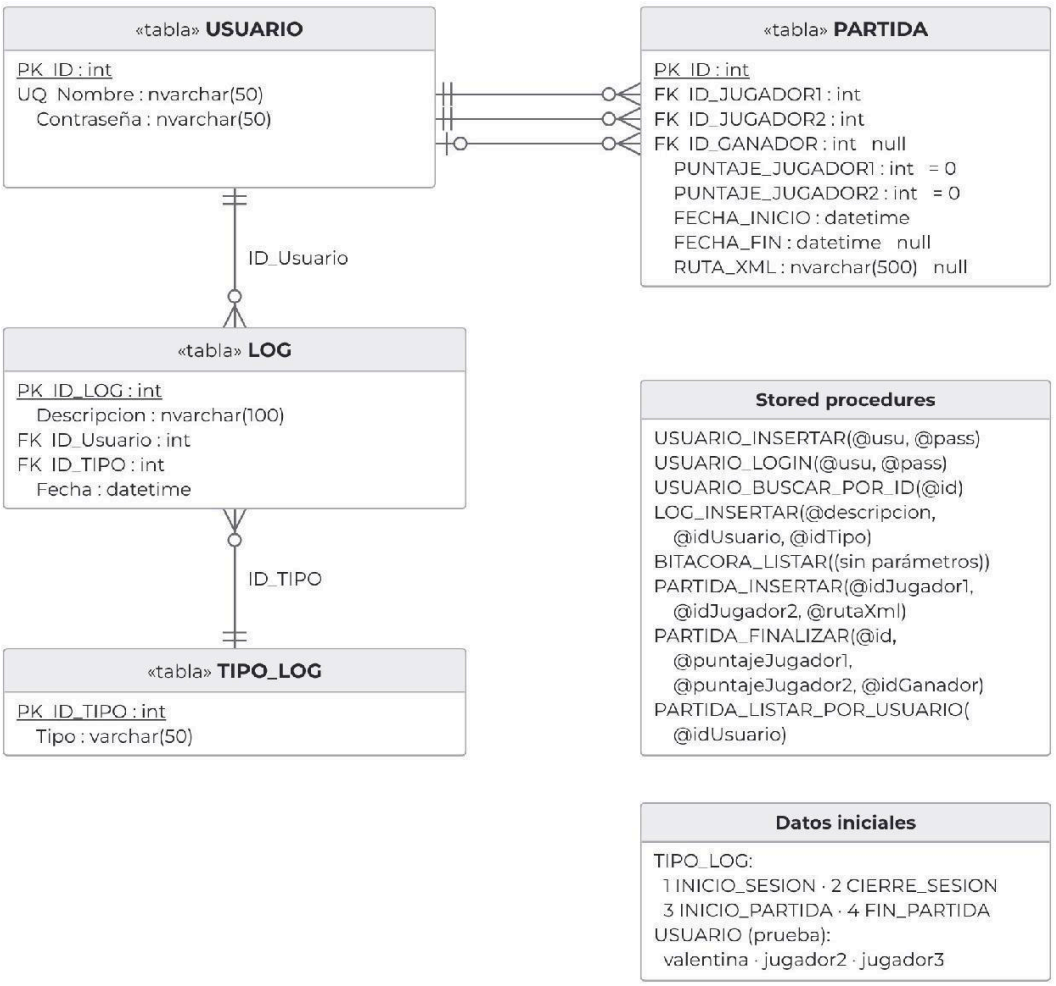


Figura 2. *DER de la base GENERALA: tablas, claves, relaciones (notación pata de gallo) y stored procedures*

Tabla 2. Tablas de la base

Tabla	Contenido
USUARIO	Jugadores registrados. El nombre de usuario es único (`UQ_USUARIO_NOMBRE`).
TIPO_LOG	Tipos de evento de bitácora: INICIO_SESION, CIERRE_SESION, INICIO_PARTIDA, FIN_PARTIDA.
LOG	Bitácora: descripción, usuario, tipo de evento y fecha/hora.
PARTIDA	Jugadores, ganador (nulo en caso de empate), puntajes, fechas de inicio y fin y ruta del XML de movimientos.

Tabla 3. Stored procedures

Procedimiento	Uso
USUARIO_INSERTAR	Alta de usuario; devuelve el ID generado, o 0 si el nombre ya existe. Bloquea la lectura del último ID para que dos altas simultáneas no choquen.
USUARIO_LOGIN	Valida nombre y contraseña. La contraseña distingue mayúsculas y minúsculas y también los espacios al final.
USUARIO_BUSCAR_POR_ID	Obtiene un usuario (nombre del rival en estadísticas).
LOG_INSERTAR	Agrega un registro a la bitácora con la fecha actual.
PARTIDA_INSERTAR	Crea la partida con su fecha de inicio y la ruta del XML (hasta 500 caracteres).
PARTIDA_FINALIZAR	Guarda puntajes, ganador y fecha de fin. No modifica una partida que ya estaba cerrada.
PARTIDA_LISTAR_POR_USUARIO	Partidas terminadas de un jugador, para las estadísticas.
BITACORA_LISTAR	Lista la bitácora con el nombre del usuario y del tipo de evento, del más nuevo al más viejo, para la pantalla Bitácora.

7. Funcionalidades implementadas

7.1 Juego

- Generala completa para 2 jugadores, con hasta 3 tiradas por turno y conservación individual de cada dado.
- Cálculo automático de las 11 categorías, con bonus por tirada servida en las combinaciones.
- Categorías sin puntaje tachadas (se anota 0, previa confirmación) y sin posibilidad de reutilizarse.
- Tachado a voluntad con el botón *Tachar*: se anota 0 en cualquier categoría disponible aunque los dados den puntos.
- El selector de categorías muestra, después de cada tirada, cuánto daría cada categoría con los dados actuales, y si es servida o queda tachada.
- Indicador del turno del jugador (“Turno X de 11”) y de quién va ganando y por cuánto.
- Cambio automático de turno, determinación de ganador o empate y abandono de partida eligiendo qué jugador abandona.
- Atajos de teclado: T tira los dados y Enter anota la categoría elegida.

7.2 Usuarios y sesión

- Registro de usuarios con credenciales propias y nombre de usuario único, con reglas de alta: nombre de 3 a 20 caracteres sin espacios y contraseña de 4 a 20 caracteres con al menos un número.
- Login de ambos jugadores y menú principal después del login (Jugar, Estadísticas, Bitácora, Reglas y Cerrar sesión). El logout de los dos se registra en la bitácora al elegir *Cerrar sesión* o al cerrar la aplicación desde el menú; en ambos casos se pide confirmación.

7.3 Registro y persistencia

- Bitácora de inicio/cierre de sesión e inicio/fin de partida para cada jugador, consultable desde el menú (ver sección 8.1).
- Archivo XML propio por partida con cada tirada, cada anotación y el cierre, consultable desde Estadísticas → Ver movimientos y descargable con Descargar XML (ver sección 8.2).
- Estadísticas por jugador: ganadas, perdidas, empatadas, promedio de victorias y tiempo total jugado, con el historial filtrable por resultado.
- Backup y Restore de la base GENERALA desde la propia aplicación, con confirmación antes de restaurar y deshabilitados durante una partida.

7.4 Robustez

- Los errores de la base de datos no se ocultan: la BLL los convierte en excepciones con un mensaje claro y cada formulario los muestra. Además, un manejador global evita que un error inesperado cierre la aplicación.
- Las conexiones se cierran siempre (`try/finally`) aunque falle la consulta.
- El inicio y el cierre de cada partida se graban en una transacción junto con la bitácora: si algo falla se hace rollback y no queda nada a medias.
- Si no se puede guardar el resultado de la partida (al completarla o al abandonarla), `BLL.PARTIDA` no la cierra en memoria: la partida queda en curso, se muestra el error y se puede reintentar. Si el abandono falla al volver al menú, la pantalla de juego no se cierra.
- El cierre de sesión de cada jugador se registra por separado: si falla el del Jugador 1, igual se intenta el del Jugador 2, y el mensaje de error indica de quién falló.
- Los stored procedures que dan de alta calculan el ID con bloqueo (`UPDLOCK, HOLDLOCK`), así dos altas simultáneas no obtienen el mismo ID.
- El restore pone la base en SINGLE_USER y restaura en un solo comando; después vacía el pool de conexiones y la aplicación vuelve al login. Si falla, la base vuelve a modo multiusuario y no queda inutilizable; la ruta del archivo se escapa antes de armar el comando.
- Los diálogos de backup y restore arrancan en la carpeta de backups de SQL Server (donde el servicio tiene permisos) y, si algo falla, muestran el mensaje real del motor.
- Conexión automática: la DAL prueba el Data Source de `GUI/App.config`, `.\SQLEXPRESS` y la instancia por defecto (`.`), y recuerda la que funcionó. Para usar otra instancia alcanza con cambiar el Data Source de las cadenas GENERALA y MASTER, sin recompilar.

7.5 Pruebas unitarias

El proyecto `Tests.Unitarios` (MSTest, .NET Framework 4.7.2) prueba la lógica de la BLL que no necesita base de datos, así se puede comprobar en segundos que un cambio no rompió el cálculo de puntajes ni las reglas de alta. Se ejecuta desde Visual Studio con **Prueba → Ejecutar todas las pruebas** o desde el Explorador de pruebas.

- `GeneralaTests` (11 pruebas): puntaje de Unos, Escalera de 1 a 5 y de 2 a 6, escalera incompleta (0), Full, Póker, Generala y Generala Doble; la tirada servida duplica la Escalera pero no los números; y tirar con los 5 dados guardados no los cambia.
- `UsuarioTests` (4 pruebas): `ValidarRegistro` acepta un usuario válido y rechaza un nombre con espacios, una contraseña sin números y una contraseña demasiado corta.

8. Bitácora, XML de movimientos y estadísticas

8.1 Bitácora

El inicio y el cierre de sesión se registran con `BLL.BITACORA.Registrar`, que usa `DAL.MP_LOG`. El inicio y el fin de partida los graba `DAL.MP_PARTIDA` dentro de la misma transacción que la partida, para ambos jugadores. En todos los casos se inserta en la tabla LOG mediante `LOG_INSERTAR`.

ID	Tipo	Cuándo se registra
1	INICIO_SESION	Login exitoso del Jugador 1 o del Jugador 2.
2	CIERRE_SESION	*Cerrar sesión* en el menú, cierre de la aplicación desde el menú o reemplazo del Jugador 2 por otro usuario.
3	INICIO_PARTIDA	Al presionar *Comenzar Partida*.
4	FIN_PARTIDA	Al completarse las 11 categorías, al abandonar o al volver al menú con una partida en curso.

Para consultarla, el botón **Bitácora** del menú abre `FormBitacora`, que muestra todos los registros (fecha y hora, usuario, evento y descripción), del más nuevo al más viejo. Los datos llegan por `BLL.BITACORA.Listar` → `DAL.MP_LOG.Listar` → `BITACORA_LISTAR`, y el formulario solo los muestra.

8.2 Registro de movimientos en XML

Al iniciar cada partida, `MOVIMIENTOXML` crea un archivo en la carpeta **Partidas** junto al ejecutable (al correrlo desde Visual Studio, `GUI\bin\Debug\Partidas`), con el nombre `Partida_aaaaMMdd_HHmmssfff.xml` y los nombres de los dos jugadores, y su ruta queda guardada en `PARTIDA.RUTA_XML` para poder ubicarlo después. Durante la partida se agregan tres tipos de nodos: **Tirada** en cada tirada de dados (número de tirada del turno, dados que salieron y posiciones de los dados que se guardaron), **Movimiento** cada vez que un jugador anota una categoría, y **Fin** al cerrar la partida (puntajes, ganador y, si corresponde, quién abandonó):

```
<Partida fecha="10/10/2026 18:42:07" jugador1="valentina" jugador2="jugador2">
  <Tirada turno="1" jugador="valentina" numero="1" dados="3,3,1,5,6"
          guardados="" fecha="10/10/2026 18:42:15" />
  <Tirada turno="1" jugador="valentina" numero="2" dados="3,3,3,5,5"
          guardados="1,2,4" fecha="10/10/2026 18:42:24" />
  <Movimiento turno="1" jugador="valentina" dados="3,3,3,5,5"
              categoria="Full" puntaje="30" fecha="10/10/2026 18:42:31" />
  <Tirada turno="2" jugador="jugador2" numero="1" dados="1,2,3,4,5"
          guardados="" fecha="10/10/2026 18:42:50" />
  <Movimiento turno="2" jugador="jugador2" dados="1,2,3,4,5"
              categoria="Escalera" puntaje="40" fecha="10/10/2026 18:42:58" />
  ...
  <Fin turnos="22" jugador1="valentina" puntajeJugador1="215"
       jugador2="jugador2" puntajeJugador2="198" ganador="valentina"
       abandonada="no" abandono="" fecha="10/10/2026 19:05:12" />
</Partida>
```

En el ejemplo, valentina guarda los dados 1, 2 y 4 para la segunda tirada y anota Full; la anotación de jugador2 es una escalera servida (20 × 2), lograda en la primera tirada del turno.

Para su posterior consulta, en Estadísticas se elige una partida del historial y el botón Ver movimientos lee el archivo con `MOVIMIENTOXML.LeerMovimientos` y lo muestra en `FormMovimientos`, en orden: las tiradas, las anotaciones en negrita y el cierre resaltado al final. Si el archivo ya no existe o está dañado, la aplicación lo avisa.

La base guarda la ruta completa del XML en la PC donde se jugó, así que en otra PC esa ruta puede no existir (por ejemplo, si la solución se copió a otra carpeta o se restauró un backup en otra máquina). Por eso, antes de abrirlo, `BLL.MOVIMIENTOXML.Resolver` prueba primero la ruta guardada y, si no la encuentra, busca el mismo nombre de archivo en la carpeta **Partidas** de la aplicación actual. Si tampoco está ahí, se avisa que no se encontró el XML. En `FormMovimientos` se muestra solo el nombre del archivo, y la ruta completa aparece al pasar el mouse.

Desde `FormMovimientos`, el botón **Descargar XML** guarda una copia del archivo original de la partida en la carpeta que elija el usuario, sin modificarlo; si el archivo ya no existe, la aplicación lo avisa. La aplicación no importa archivos XML: el registro se genera solo durante la partida y se usa únicamente para consulta.

Con esto queda cubierto el requisito de la consigna de registrar en un XML todos los movimientos de la partida para su posterior consulta: cada partida tiene su archivo, se puede ver desde la aplicación en cualquier momento y se puede llevar fuera de ella.

8.3 Estadísticas

`FormEstadisticas` se abre desde el menú o desde la pantalla de juego y permite elegir a cualquiera de los dos jugadores logueados. `BLL.ESTADISTICA` arma su historial de partidas terminadas (fecha, rival, puntaje propio, puntaje del rival y resultado) y calcula los siguientes indicadores, que el formulario solo muestra:

Indicador	Cálculo
Ganadas / Perdidas / Empatadas	Conteo de partidas según `ID_GANADOR` (nulo = empate).
Promedio de victorias	Ganadas ÷ total de partidas jugadas × 100, expresado en porcentaje.
Tiempo total jugado	Suma de (fecha de fin − fecha de inicio) de todas las partidas, en horas, minutos y segundos.

El historial se puede filtrar por resultado (Todas, Ganadas, Perdidas o Empates). El filtro lo aplica `BLL.ESTADISTICA.Filtrar` sobre el historial ya cargado, y los contadores no cambian: siempre reflejan todas las partidas del jugador.

9. Diagramas de clases

Diagramas de la solución completa, generados a partir del código fuente. Cada capa tiene su color (GUI rosa, BLL violeta, DAL celeste y BE verde) y cada clase muestra su estereotipo («Form», «UserControl», «static», «internal», «delegate» o «enumeration»). Los miembros estáticos y las constantes van subrayados y los abstractos en cursiva; + - # ~ indican público, privado, protegido e internal.

Las relaciones se dibujan con líneas ortogonales que no se cruzan ni pasan por encima de las clases: herencia (triángulo), asociación (línea continua con flecha), composición FormPrincipal–ucDado (rombo relleno, 5 dados) y dependencia (línea punteada). Las flechas que van al mismo destino se unen con un punto. Las clases de otra capa que usa cada clase se listan en su pie (BE:, BLL:, DAL:; el asterisco indica que es un atributo) y las utilidades Mensaje y Tema en «Usa:». En los mappers, «T = BE.X» indica la entidad con la que cada uno hereda de MAPPER<T>.

La Figura 3 resume la arquitectura; las Figuras 4 a 7 muestran cada capa por separado (en GUI, los formularios con sus atributos; los métodos completos están en la Figura 8), y la Figura 8 es el diagrama completo con todas las clases, atributos, métodos, parámetros y tipos. Por su tamaño, conviene verla con zoom. La Figura 9 muestra la navegación entre formularios: cada flecha es un formulario que se abre como ventana modal y el rótulo es el botón que lo abre.

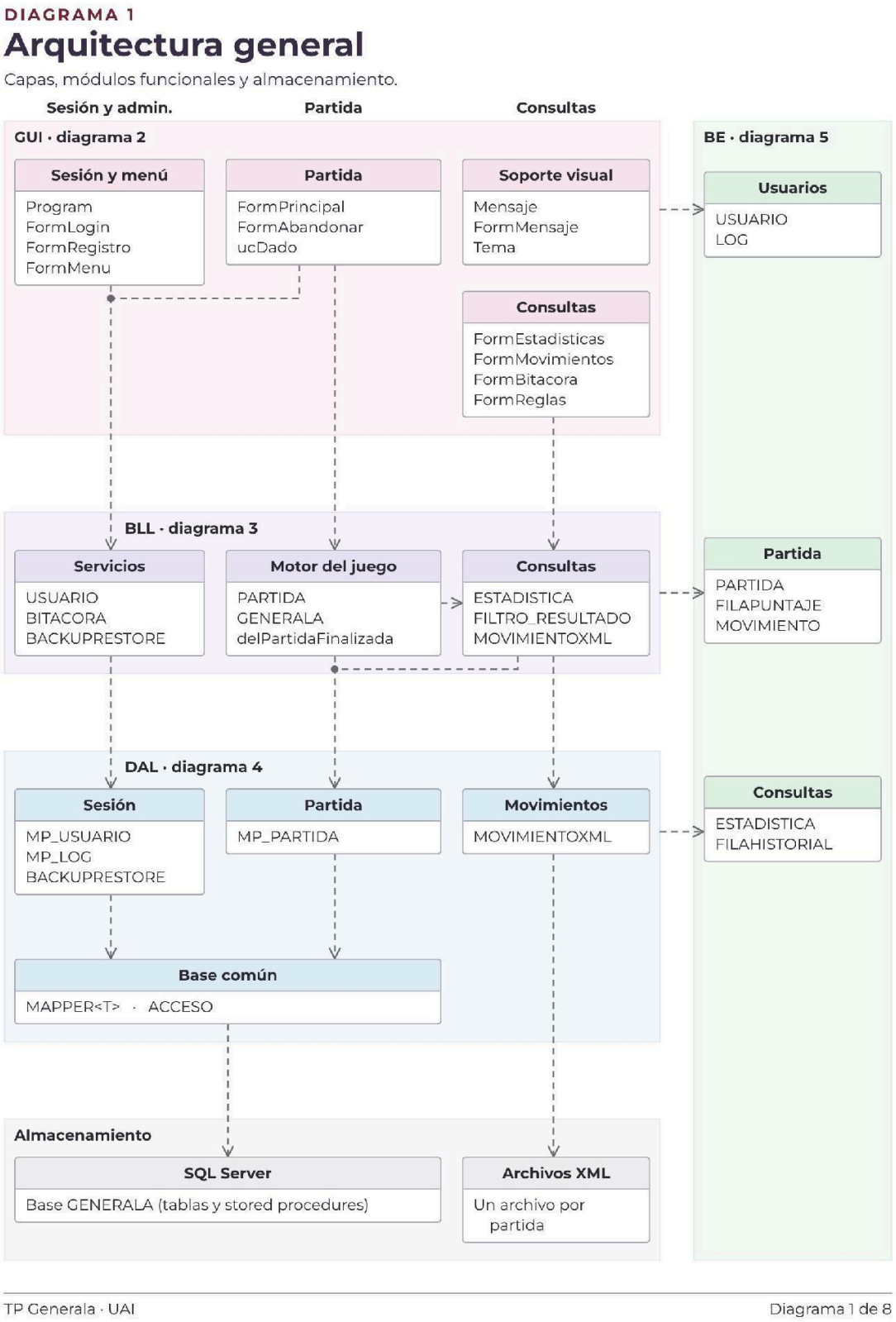


Figura 3. *Arquitectura general: capas, módulos funcionales y almacenamiento*

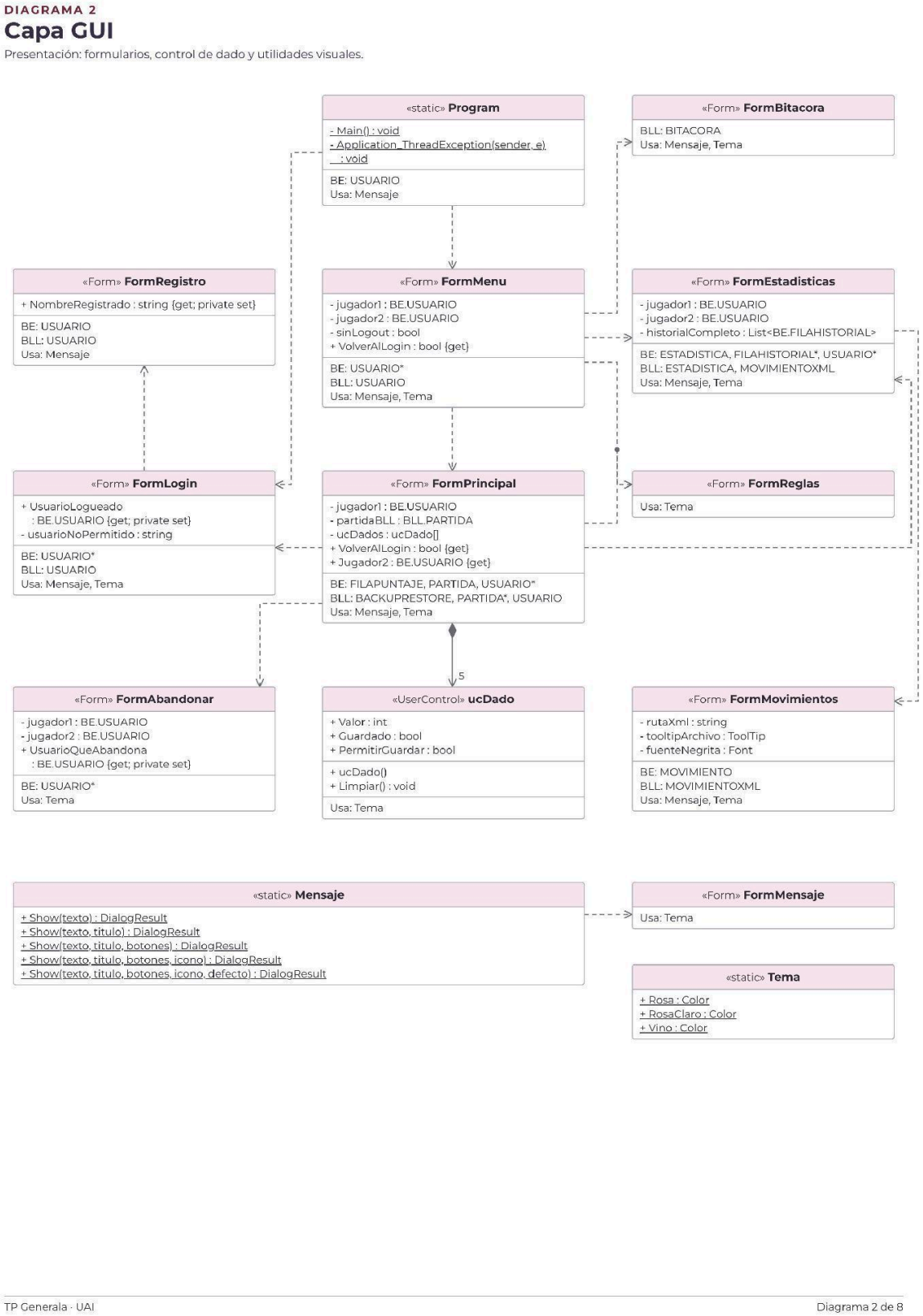


Figura 4. *Clases de la capa GUI y sus relaciones*

DIAGRAMA 3

Capa BLL

Lógica de negocio: reglas de la Generala, partida, usuarios, estadísticas y bitácora.

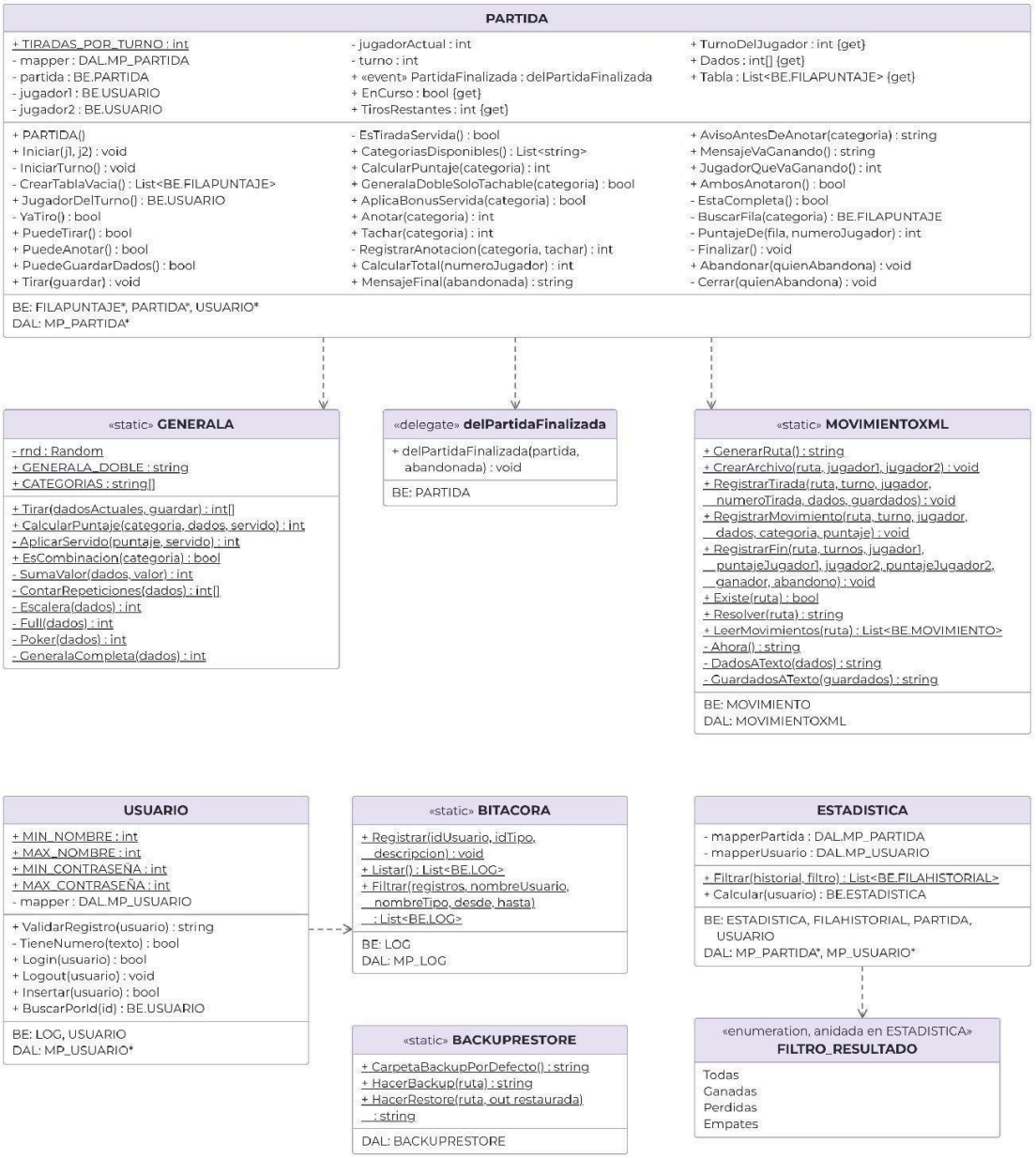


Figura 5. *Clases de la capa BLL y sus relaciones*

DIAGRAMA 4

Capa DAL

Acceso a datos: SQL Server con stored procedures y archivos XML de movimientos.

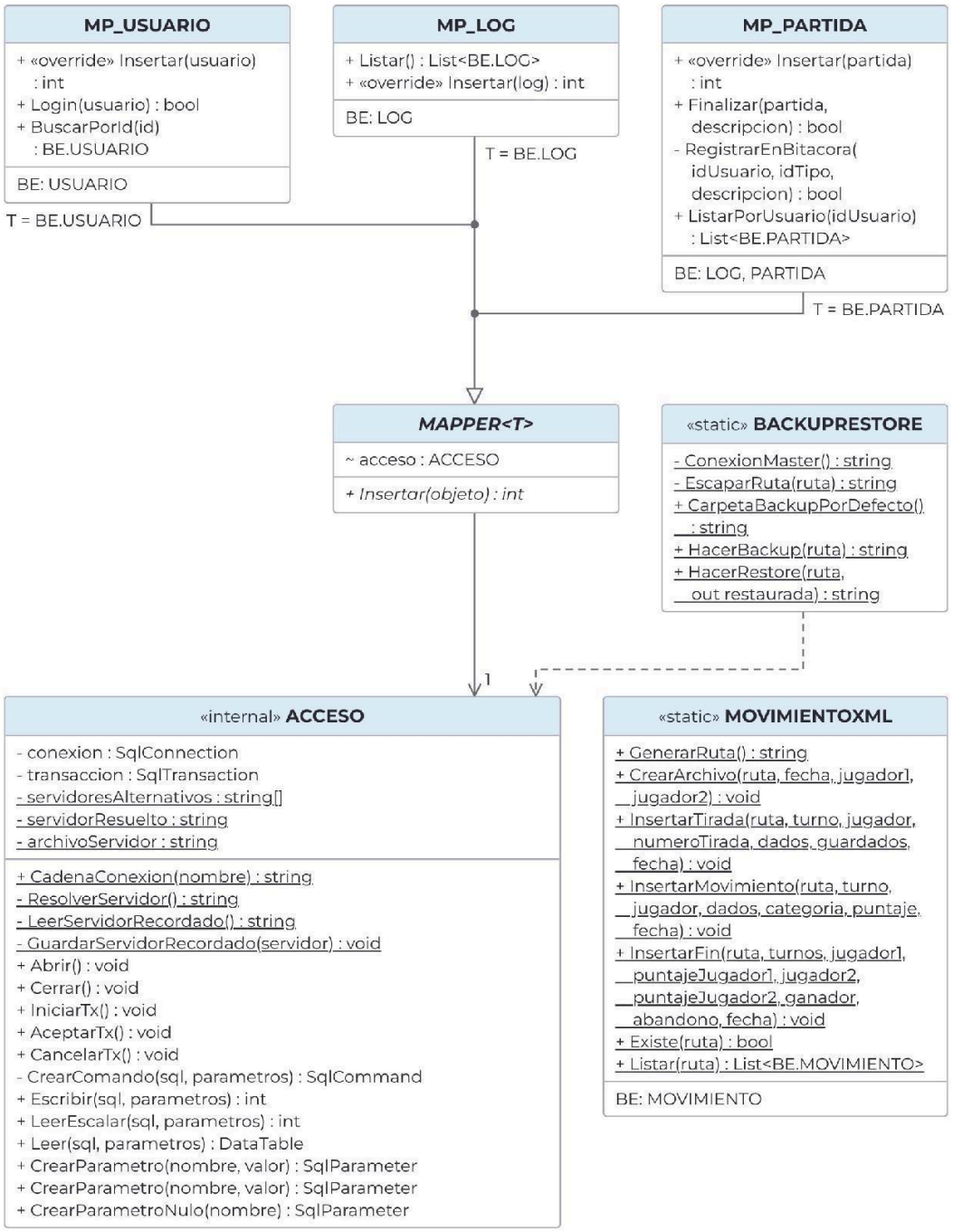
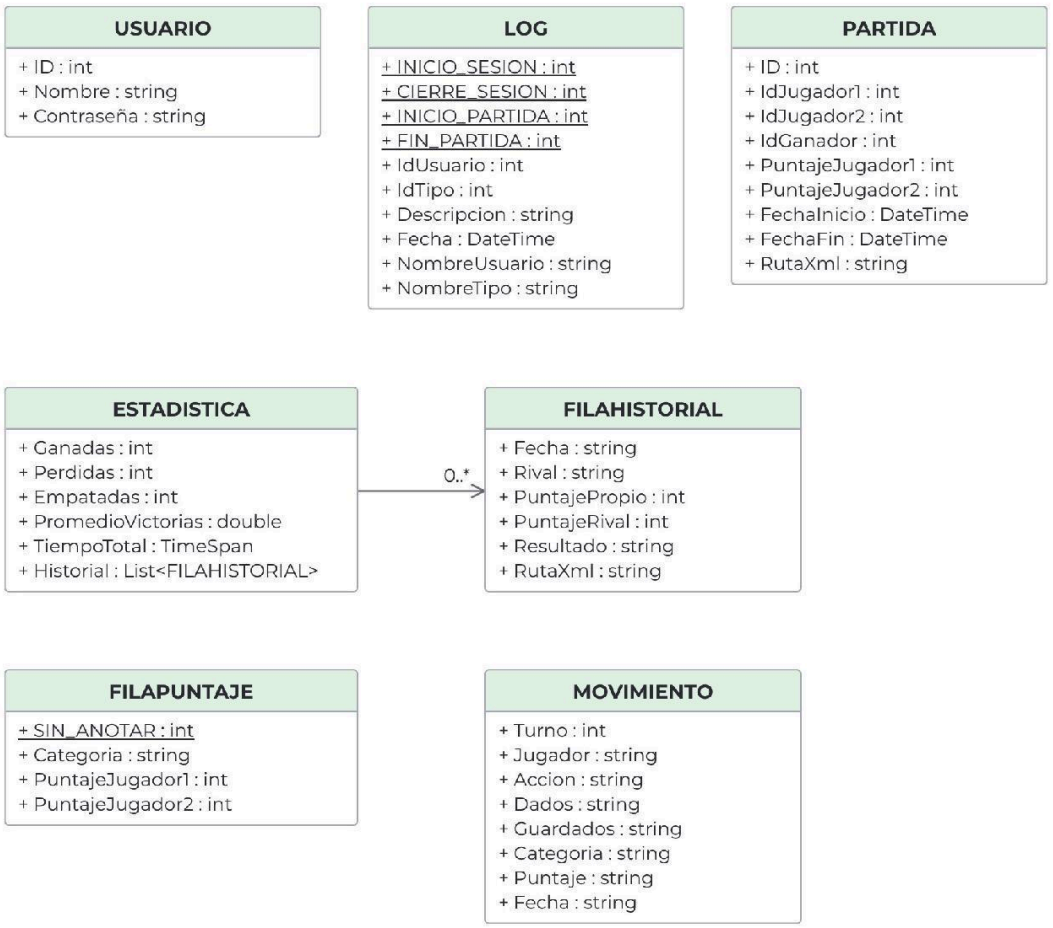


Figura 6. *Clases de la capa DAL y sus relaciones*

DIAGRAMA 5

Capa BE

Entidades: datos que comparten todas las capas.



TP Generala · UAI Diagrama 5 de 8

Figura 7. *Clases de la capa BE y sus relaciones*

DIAGRAMA 6

Diagrama de clases

Solución completa: las 36 clases con todos sus atributos, métodos y parámetros.

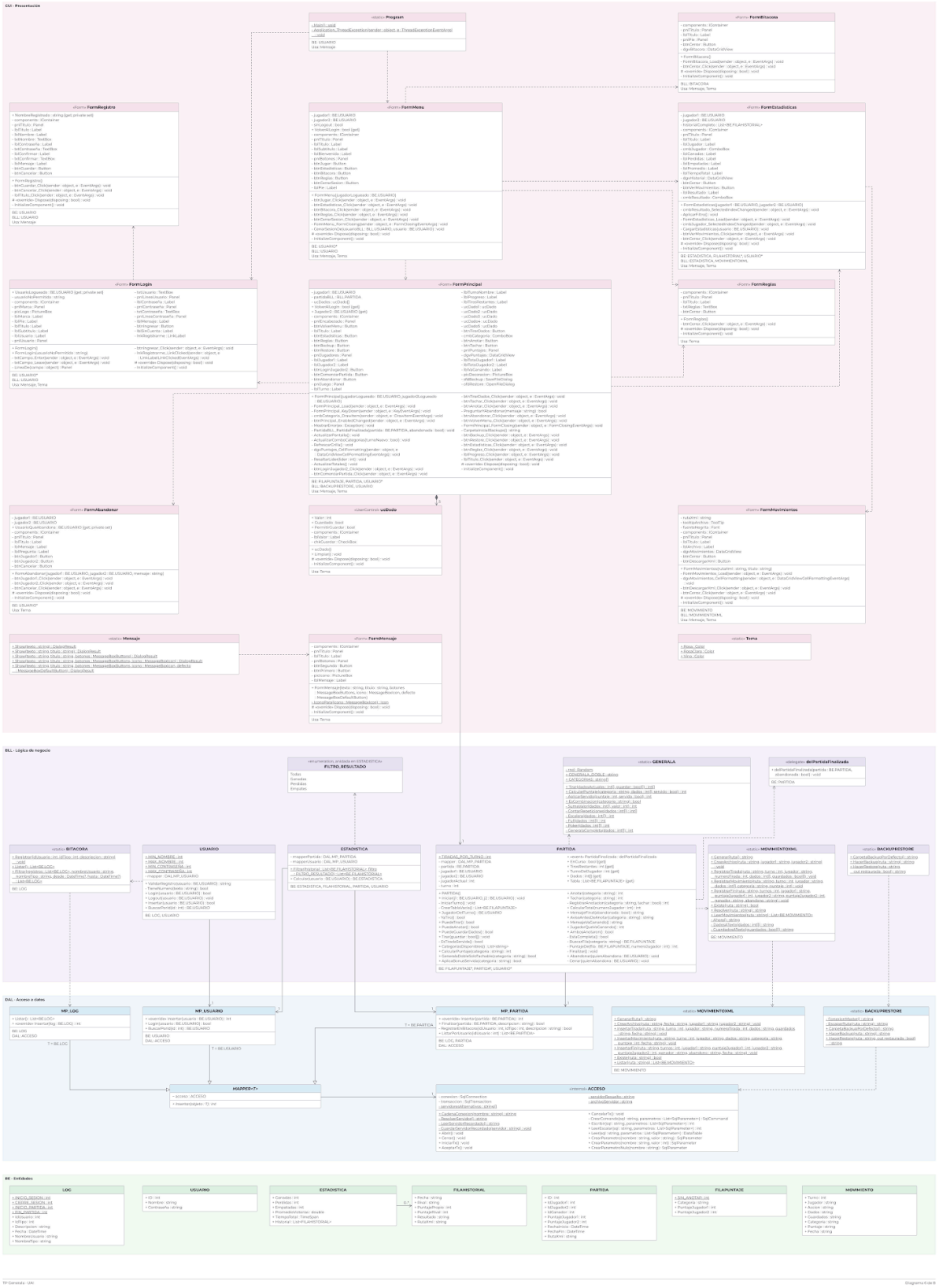


Figura 8. *Diagrama de clases completo: todas las clases con sus atributos, métodos y parámetros*

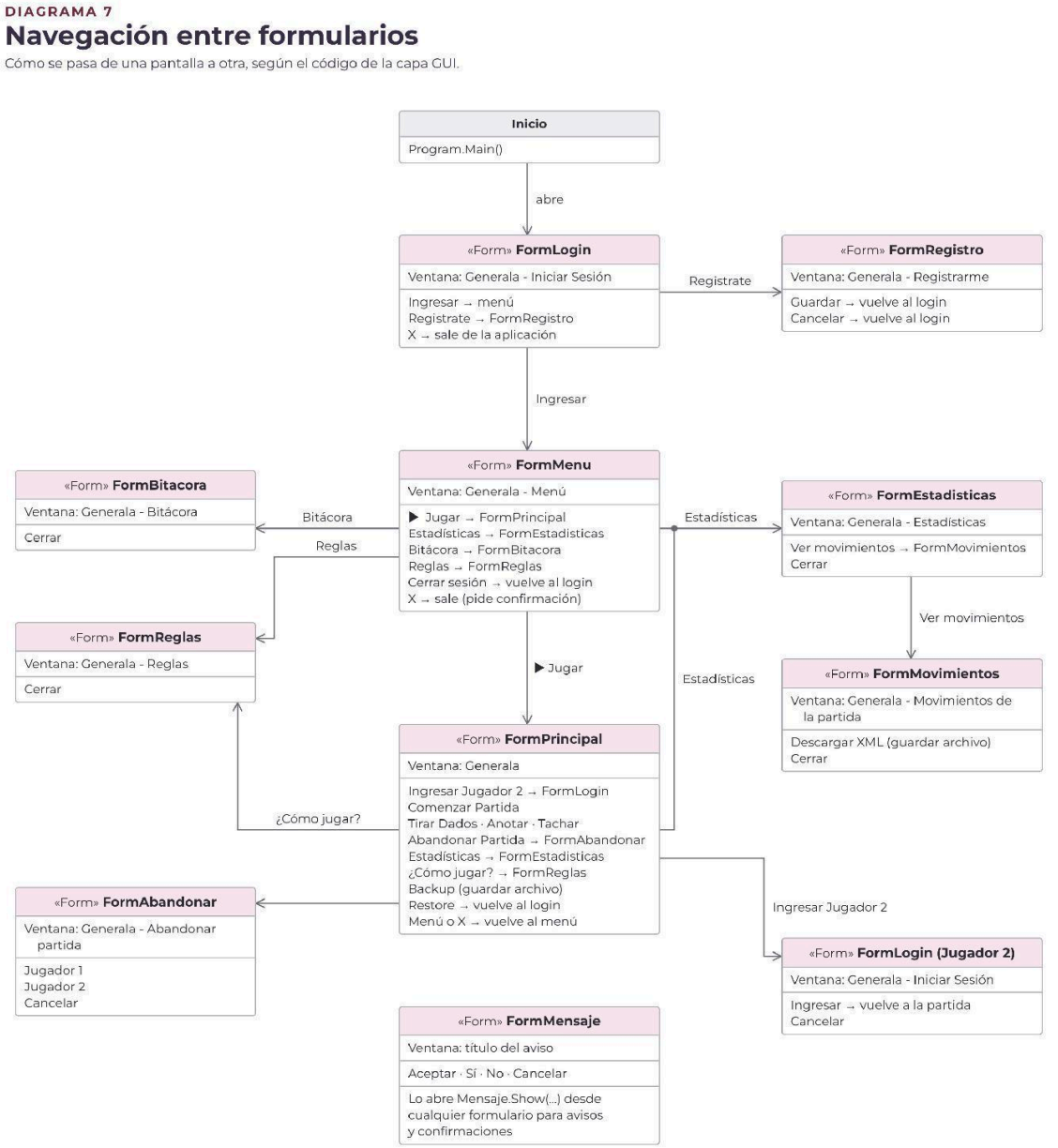


Figura 9. *Navegación entre formularios*

10. Imágenes del sistema

Capturas de la aplicación en funcionamiento, en el orden en que se usan.

10.1 Inicio de sesión

El Jugador 1 inicia sesión al abrir la aplicación. Desde el enlace Registrate se da de alta un usuario nuevo.

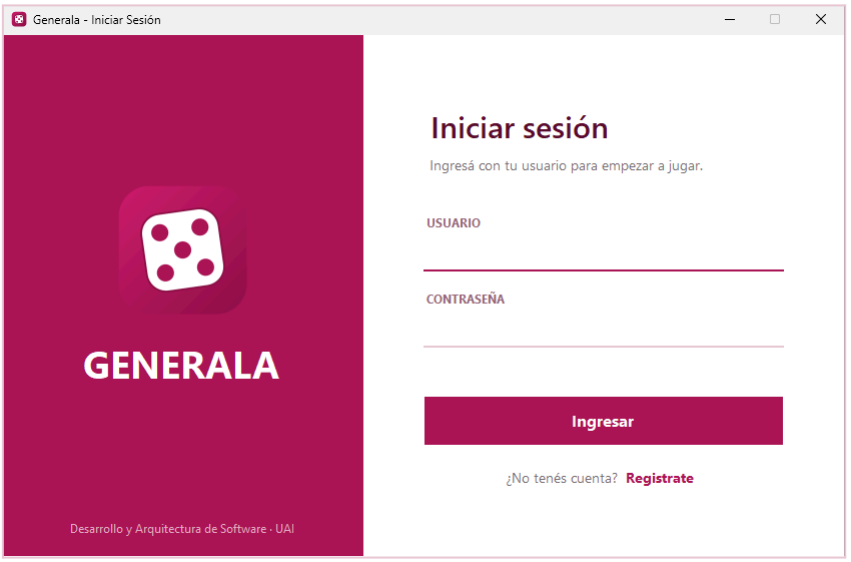


Figura 10. *Pantalla de inicio de sesión*

10.2 Menú principal

Después del login se abre el menú, que muestra el jugador conectado. Desde ahí se juega (Jugar), se consultan Estadísticas, Bitácora y Reglas, y se cierra la sesión.

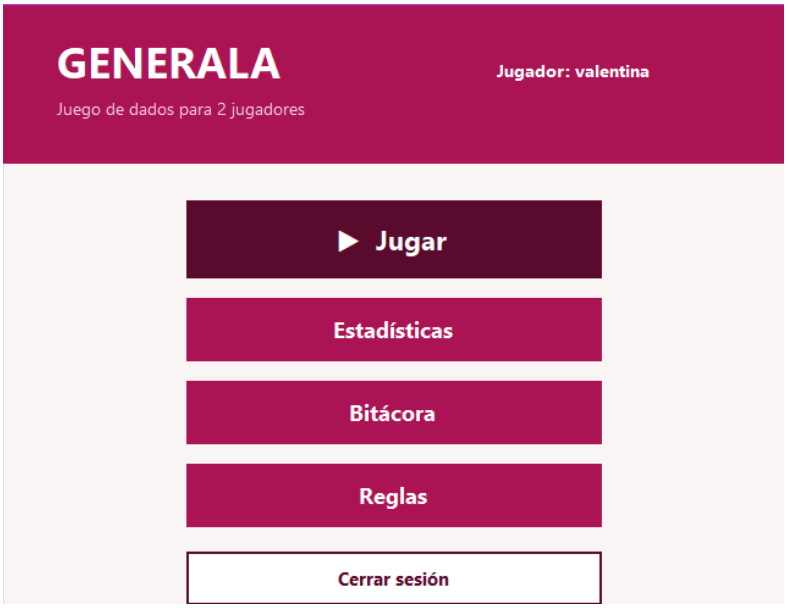


Figura 11. *Menú principal después de iniciar sesión*

10.3 Pantalla de juego

Jugar abre la pantalla de juego. Arriba están ¿Cómo jugar?, Estadísticas, Backup, Restore y Menú. Antes de empezar hay que ingresar al Jugador 2 con Ingresar Jugador 2; con los dos jugadores se habilita Comenzar Partida y el botón pasa a Cambiar Jugador 2.

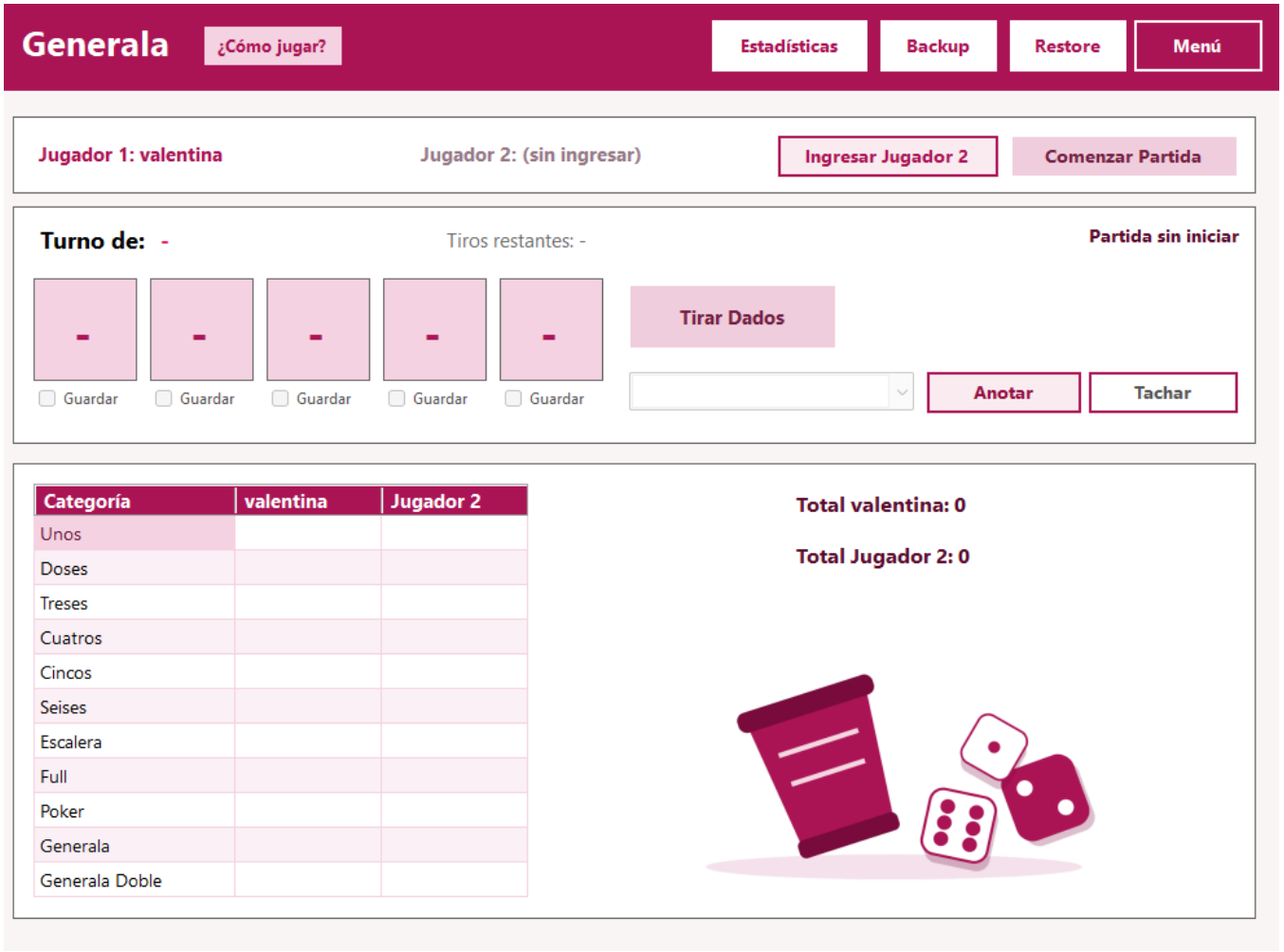


Figura 12. *Pantalla de juego antes de ingresar al Jugador 2*

10.4 Partida en curso

Durante la partida se indica el turno, las tiradas restantes, los totales y quién va ganando. El botón Comenzar Partida pasa a Abandonar Partida.

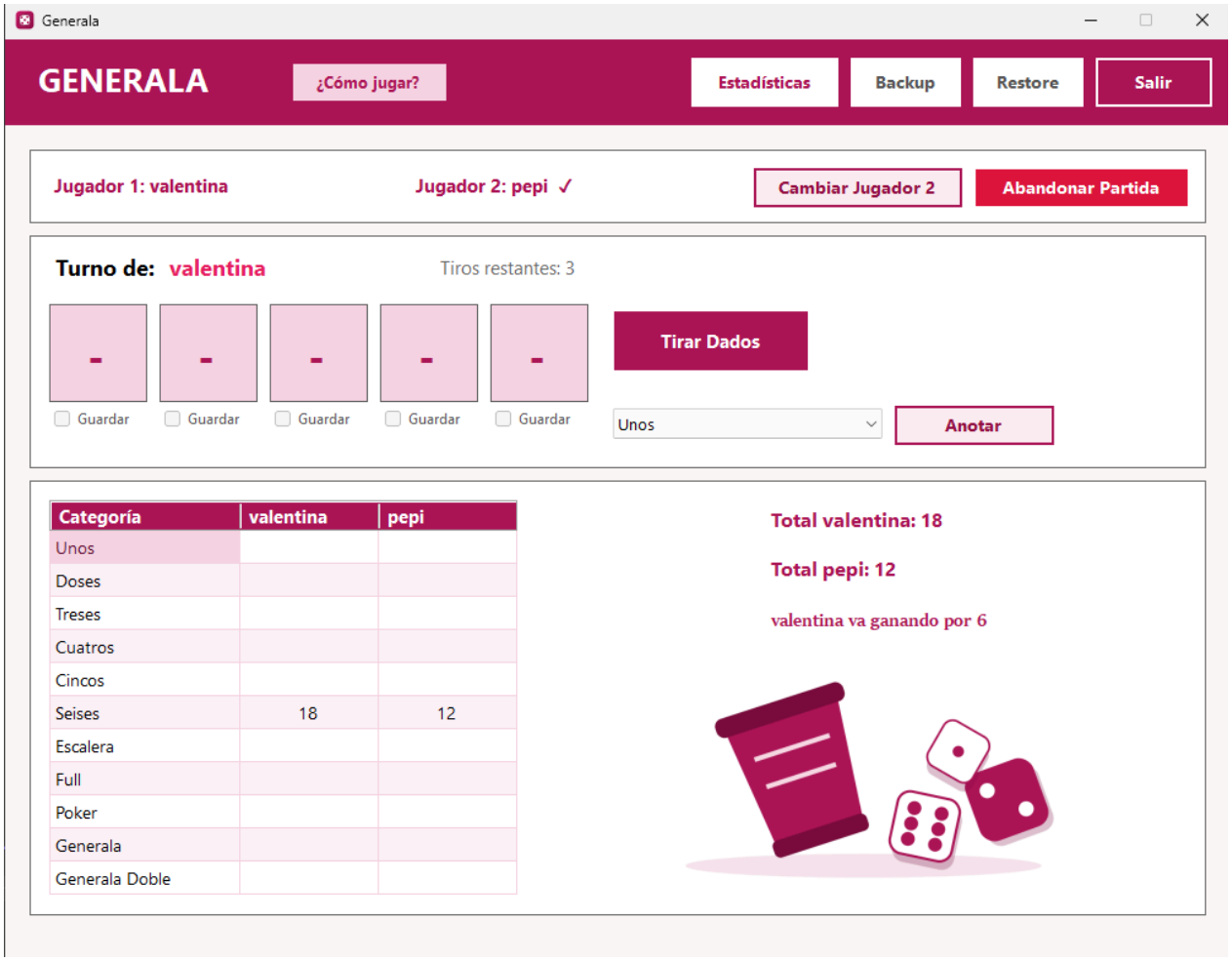


Figura 13. *Partida en curso con puntajes anotados*

10.5 Reglas del juego

La ventana de reglas resume el objetivo, la mecánica del turno, las categorías con sus puntajes y la regla de la tirada servida.



Figura 14. *Ventana de reglas del juego*

10.6 Estadísticas

Para el jugador elegido se muestran las partidas ganadas, perdidas y empatadas, el promedio de victorias, el tiempo total jugado y el historial de partidas.



Figura 15. *Estadísticas e historial de partidas del jugador*

10.7 Movimientos de la partida

Desde Ver movimientos se abre el XML de la partida elegida, con cada tirada, los dados guardados y las anotaciones de los dos jugadores.

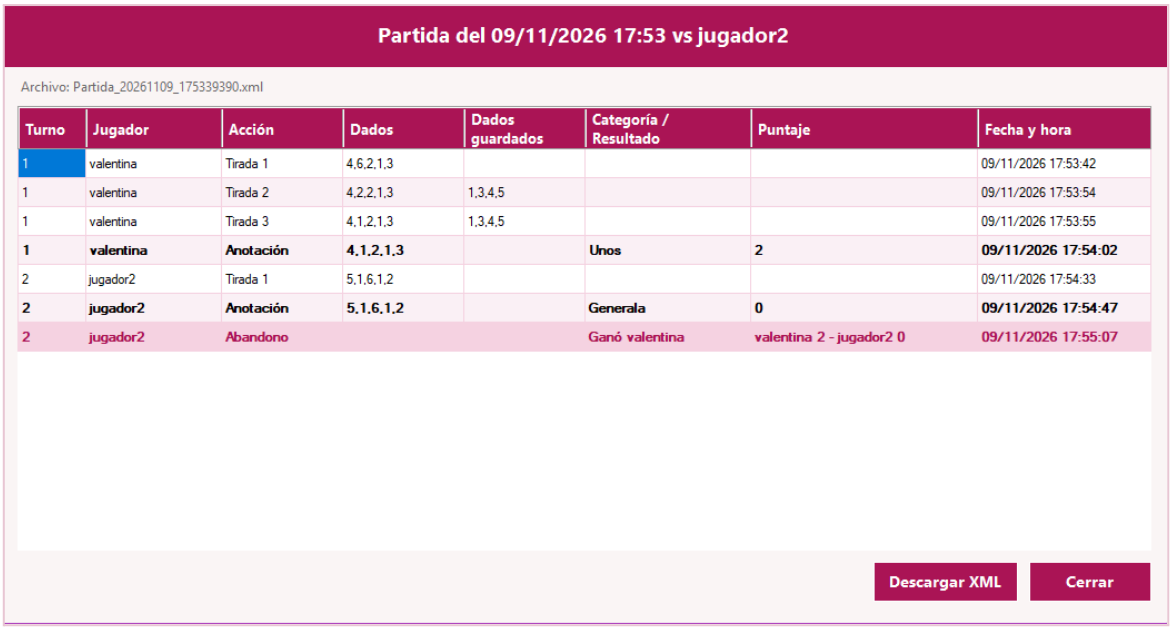


Figura 16. *Movimientos de la partida: tiradas, anotaciones en negrita y cierre*

10.8 Abandonar la partida

Al abandonar, o al volver al menú con una partida en curso, la aplicación pregunta qué jugador abandona; la partida la gana el otro. Con Cancelar, la partida sigue.



Figura 17. *Elección del jugador que abandona la partida*

10.9 Bitácora

Desde el menú, Bitácora muestra los inicios y cierres de sesión y el comienzo y fin de cada partida, del más reciente al más antiguo, con la fecha y hora, el usuario y el tipo de evento.

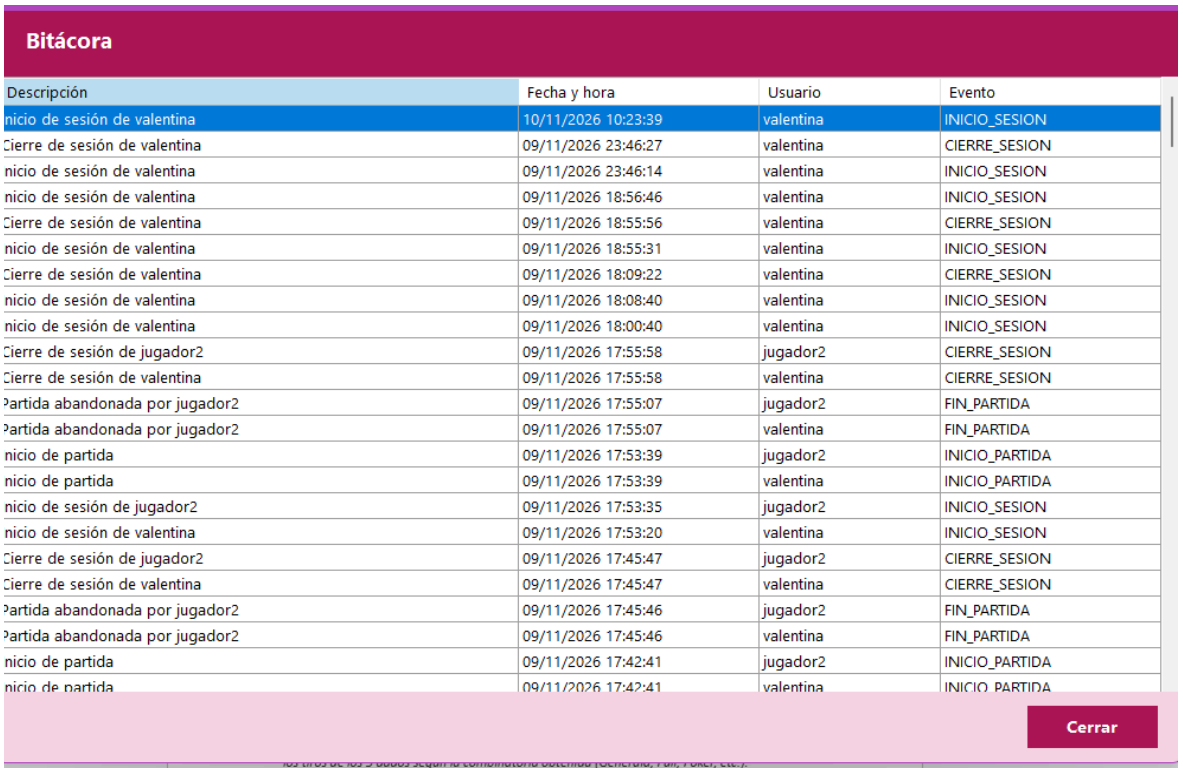


Figura 18. *Bitácora de sesiones y partidas*

10.10 Mensajes y confirmaciones

Los avisos y las confirmaciones se muestran en un cuadro propio con los colores de la aplicación (FormMensaje, que se abre con Mensaje.Show). Por ejemplo, al cerrar el menú con la X se pide confirmación antes de salir.

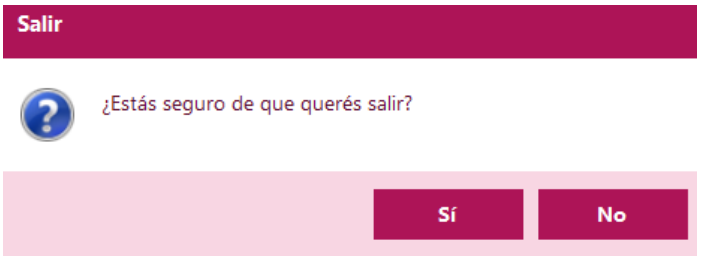


Figura 19. *Confirmación al salir de la aplicación*

11. Instalación y ejecución

11.1 Requisitos

- Visual Studio 2019 o posterior con .NET Framework 4.7.2.
- SQL Server (Express alcanza) y SQL Server Management Studio. La app busca sola la instancia (la de `App.config`, `.\SQLEXPRESS` o la instancia por defecto) y usa autenticación de Windows.
- Para el proyecto de pruebas, Visual Studio descarga los paquetes de MSTest por NuGet la primera vez que se compila.

11.2 Pasos

1. Ejecutar `Script.sql` en SQL Server Management Studio para crear la base GENERALA, sus tablas, datos iniciales y stored procedures. Si la base ya existía de una versión anterior, hay que volver a ejecutarlo: agrega el procedimiento `BITACORA_LISTAR` que usa la pantalla Bitácora.
2. Abrir `TP_Generala.sln` y establecer **GUI** como proyecto de inicio.
3. Solo si la instancia de SQL Server tiene otro nombre, cambiar el Data Source de las cadenas GENERALA y MASTER en `GUI/App.config` (o en `GUI.exe.config` si ya está compilado).
4. Compilar y ejecutar.
5. Ingresar con un usuario de prueba como Jugador 1, elegir **Jugar** en el menú e ingresar otro usuario como Jugador 2 desde *Ingresar Jugador 2*.

Tabla 6. Usuarios de prueba

Usuario	Contraseña
valentina	1234
jugador2	1234
jugador3	1234